Morphological Abduction Architecture (MAA): Overcoming Topological Local Minima in Graph-Rewriting Cognition Engines

Irfan Mahir — Independent Researcher, Dhaka, Bangladesh — irfan@furylogic.com

Contents

Morphological Abduction Architecture (MAA): Overcoming Topological Local Minima in Graph-Rewriting Cognition Engines **1**
Abstract 1
1. Introduction 1
2. Component 1: Morphological Decomposition 2
3. Component 2: Abductive Synthesis Graphing 2
4. State-Dependent Cognitive Dispatching 2
5. Conclusion 3

Morphological Abduction Architecture (MAA): Overcoming Topological Local Minima in Graph-Rewriting Cognition Engines

Author: Irfan Mahir *Independent Researcher, Dhaka, Bangladesh* irfan@furylogic.com **Date:** July 2026 **Subjects:** Artificial Intelligence (cs.AI); Computation and Language (cs.CL); Logic in Computer Science (cs.LO)

Abstract

Graph-rewriting cognition engines offer a rigid, verifiable alternative to the fluid approximations of statistical large language models. However, when these architectures treat semantic inputs as atomic, unbreakable nodes, they become highly susceptible to topological stagnation. When encountering an undefined concept (an orphan node), standard inference pipelines (deduction, induction, analogy) fail because the search space is mathematically empty, causing the system to infinitely loop its active cache. We propose the Morphological Abduction Architecture (MAA), a framework that breaks the atomic constraint by introducing Sub-Lexical Decomposition and Abductive Synthesis. By dynamically fracturing unknown nodes and mapping their sub-components to existing ontologies, the system is able to hypothesize structural meaning—transforming the engine from a rigid memorizer into a dynamic semantic synthesizer capable of state-dependent cognitive dispatching.

---

1. Introduction

In discrete graph-rewriting systems, cognition is defined by the traversal and construction of edges between known concept nodes. While this ensures strict logical adherence, it introduces a critical vulnerability: the "Atomic Node Trap."

When standard graph engines encounter a novel term—such as a previously unseen acronym or portmanteau—the parser initializes a singular, isolated node $\emptyset_V$. Because this node possesses a degree of zero ($deg(\emptyset_V) = 0$), there are no topological pathways for the inference engine to traverse. Consequently, monolithic evaluation loops (which linearly execute deduce → induce → analogize pipelines) encounter a void. Lacking a mechanism to process the unknown, the engine falls back onto its highest-activation localized structures, resulting in pathological repetition (e.g., infinitely looping a cached analogy about consciousness regardless of the prompt).

To achieve general reasoning, a system must be capable of synthesizing meaning from the unknown. The Morphological Abduction Architecture (MAA) formalizes the mechanics of "connecting the dots" by physically fragmenting orphan nodes and generating hypothesized graph structures.

2. Component 1: Morphological Decomposition

To process an unknown variable, MAA dictates that a concept node must not be treated as a fundamental particle, but as a composite structure capable of fission.

Let the user introduce an unknown string, C_{unk}. When the parser detects that $deg(C_{unk}) = 0$ within the active ontology O, the engine suspends standard inference and triggers a Decomposition Rule.

The string C_{unk} is fractured into a set of sub-lexical fragments $F = \{f_1, f_2, ..., f_n\}$ based on linguistic boundaries, syllables, or character clusters. The engine then projects these fragments against the existing ontology to identify partial topological matches:

$$M = \{ f_i \in F \mid \exists n \in O \text{ where } similarity(f_i, n) > \tau \}$$

Case Study (The Superuser Maneuver): If the engine encounters the unknown node `[sudo]`, it evaluates to $deg([sudo]) = 0$. MAA fractures the string into $F = \{su, do\}$. The engine scans the ontology and successfully binds the fragments to known concepts: $su \rightarrow [super_user]$ $do \rightarrow [execute_action]$

The engine has successfully extracted topological anchors from a previously isolated node.

3. Component 2: Abductive Synthesis Graphing

Once morphological anchors are established, the engine cannot use Deduction (which requires absolute axiomatic truth) or Induction (which requires statistical precedence). It must use **Abduction**—the generation of the most likely explanatory hypothesis.

MAA introduces a volatile edge type to the graph schema: `EDGE_HYPOTHESIZED`. Unlike axiomatic edges, hypothesized edges are temporary structural bridges used to test theoretical topologies without corrupting the permanent ontology.

Following the extraction of match set M, the Abductive Synthesis Engine generates a Hypothesis Graph Γ_H.

For the unknown node `[rdma]`, the engine fragments the node into $\{rd, ma\}$. It finds partial matches for $rd \rightarrow [remote/read]$ and $ma \rightarrow [memory/machine]$. The engine synthesizes a cluster of theoretical connections: `[rdma] --EDGE_HYPOTHESIZED--> [remote_memory] [rdma] --EDGE_HYPOTHESIZED--> [read_machine]`

By generating these volatile edges, the node C_{unk} is no longer orphaned. The search space is populated, allowing the cognitive loop to resume execution, score the hypotheses against the surrounding context, and select the highest-probability path.

4. State-Dependent Cognitive Dispatching

The final requirement of MAA is the abolition of the monolithic execution loop. A static `respond()` pipeline that applies the same sequence of rules to every input is fundamentally a memorization behavior. True cognition requires the processing architecture to adapt to the topology of the current graph state.

MAA replaces the static loop with a dynamic Cognitive Dispatcher function, Δ. Before any inference is executed, Δ evaluates the macroscopic geometry of the active graph and routes the execution thread to specialized sub-engines:

1. **State α (Topological Collision):** If the input graph introduces an edge that directly opposes an axiomatic invariant (e.g., `machine --has--> biology`), Δ routes to the **Resolution Pipeline** to isolate and reject the contradiction.
2. **State β (Orphan Detected):** If the input graph contains a node where $deg(n) = 0$ (e.g., `[rdma]`), Δ routes to the **Abductive Pipeline** (Decomposition and Synthesis) to map the unknown.
3. **State γ (Connected Query):** Only when the input graph is fully integrated into the existing ontology does Δ route to the **Standard Inference Pipeline** to deduce new paths.

$$\Delta(\Gamma_{active}) \rightarrow \begin{cases} Pipeline_{Resolution} & \text{if } \exists \text{ contradiction in } \Gamma \\ Pipeline_{Abductive} & \text{if } \exists n \in \Gamma \text{ where } deg(n) = 0 \\ Pipeline_{Standard} & \text{otherwise} \end{cases}$$

5. Conclusion

The Morphological Abduction Architecture demonstrates that the inability of discrete graph engines to handle unknown variables is not a limitation of discrete mathematics, but a failure of parsing granularity and execution routing. By allowing the engine to fracture concepts into their morphological roots and abductively synthesize hypothesized edge relationships, we eliminate topological local minima. Furthermore, implementing a State-Dependent Cognitive Dispatcher ensures that the machine actively "reads" its own structural state before acting, permanently transitioning the architecture from a rigid data-retrieval script into an adaptive, synthesizing cognition engine.